



湖南大学  
HUNAN UNIVERSITY

# 第八章 动态规划

## Dynamic Programming

—— 湖南大学计算机学院 ——

## 小班讨论



分成六组:

第1-2组, 《算法导论》第4章思考题4-2、第15章思考题15-10

第3-4组, 《算法导论》第6章思考题6-2、第15章思考题15-4

第5-6组, 《算法导论》第8章思考题8-3、第15章思考题15-2

# 目录

## 第一节

斐波那契序列

## 第二节

动态规划法的设计思想

## 第三节

三角数塔问题

## 第四节

最大子段和（最大子数组）

## 第五节

适用动态规划法的场景

## 第六节

最长公共子序列

## 第七节

0-1背包问题

## 第八节

矩阵乘法



$$F(n) = \begin{cases} 0 & n = 0 \\ 1 & n = 1 \\ F(n-1) + F(n-2) & n \geq 2 \end{cases}$$

斐波那契序列

# 斐波那契序列



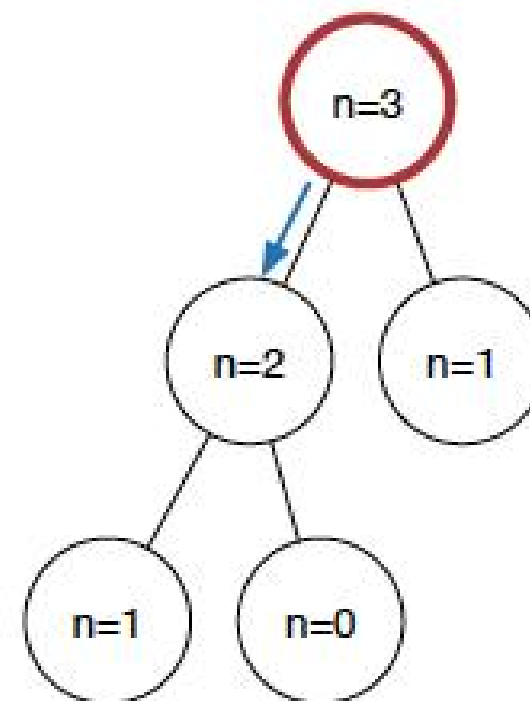
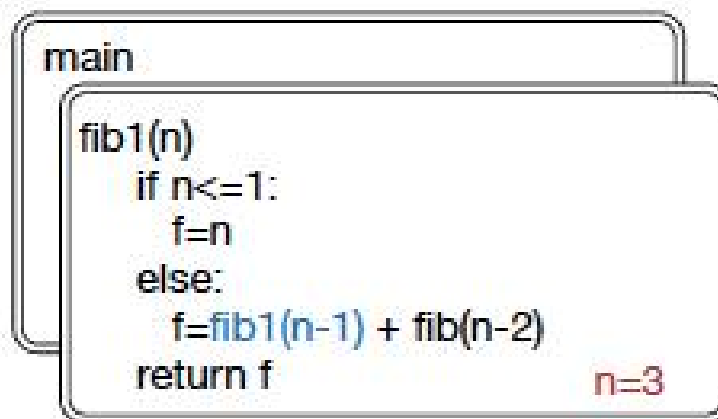
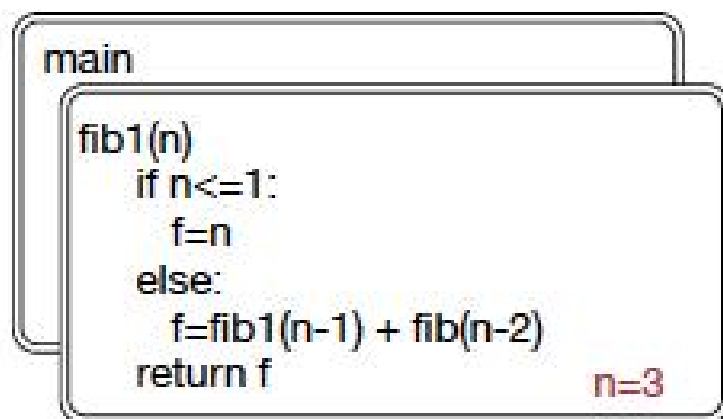
## 程序代码

```
def fib_rec(n):  
    if n <= 1:  
        f=n  
    else:  
        f=fib_rec(n-1)+fib_rec(n-2)  
    return f  
  
def main():  
    num = 5  
    print('{0:5}<=>{1:10d}'.format('fib('+str(num)+')', fib_rec(num)))  
  
if __name__ == "__main__":  
    main()
```

## 斐波那契序列



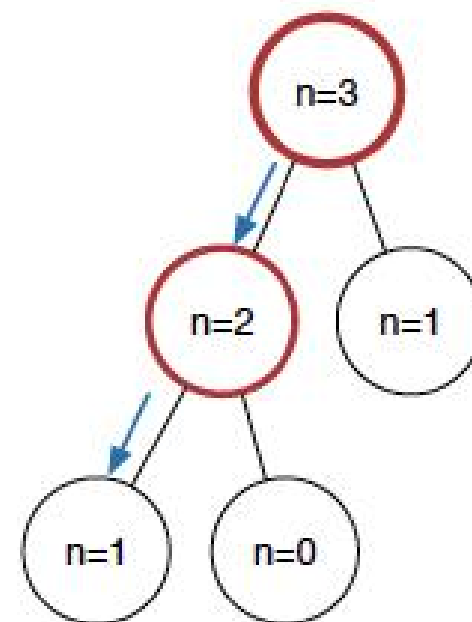
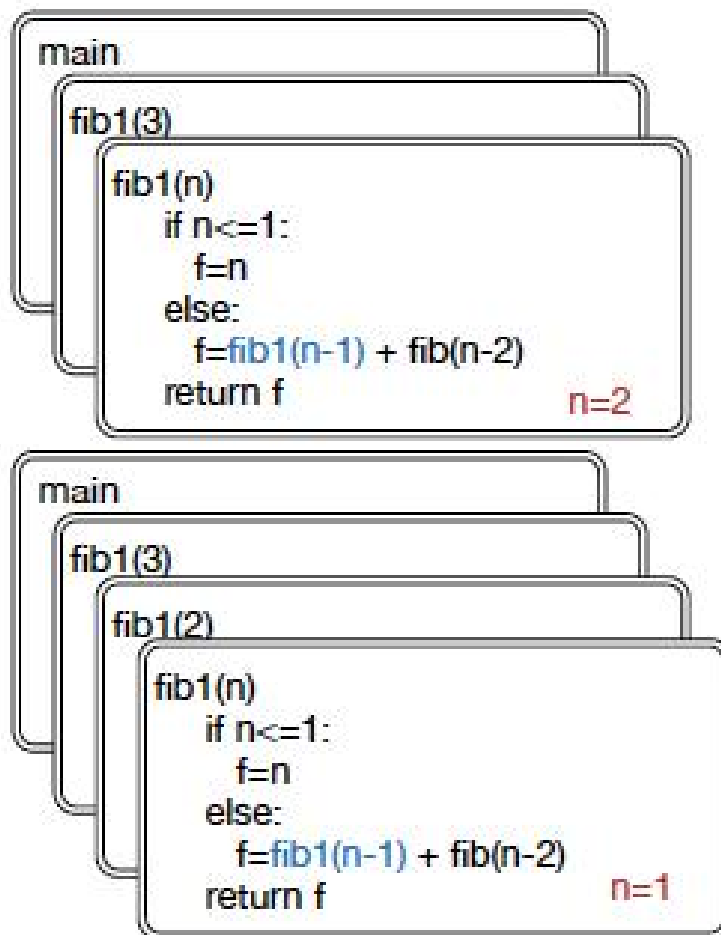
跟踪递归函数的  
执行过程



# 斐波那契序列



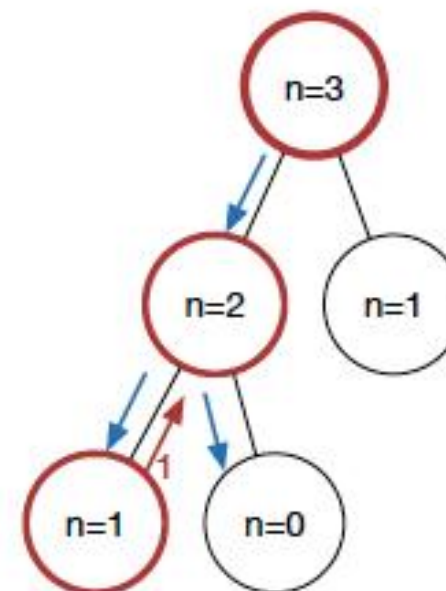
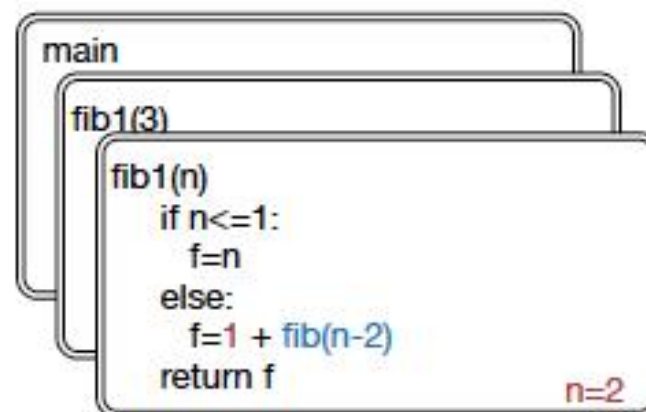
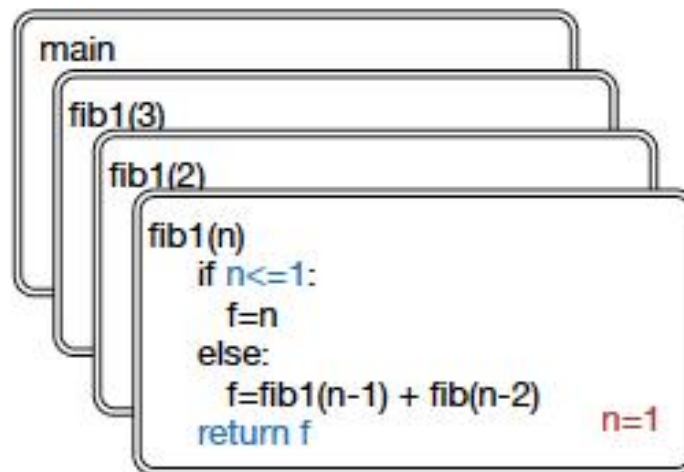
跟踪递归函数的  
执行过程



# 斐波那契序列



跟踪递归函数的  
执行过程

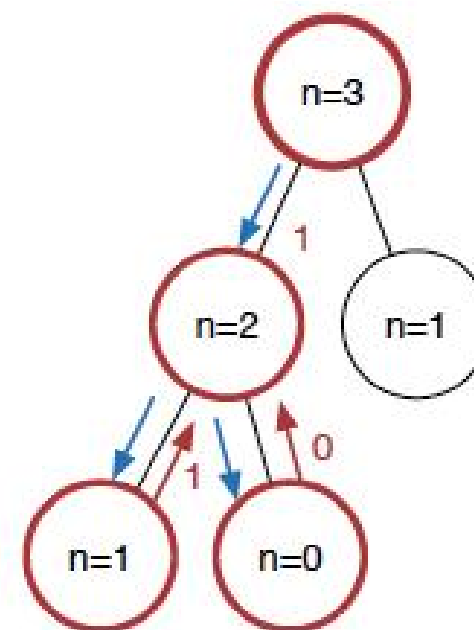
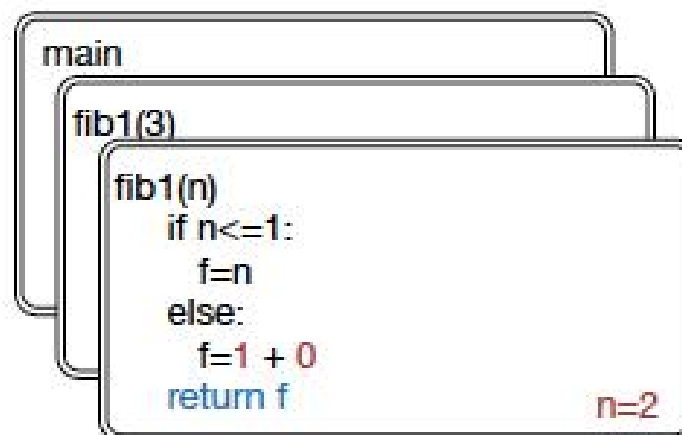
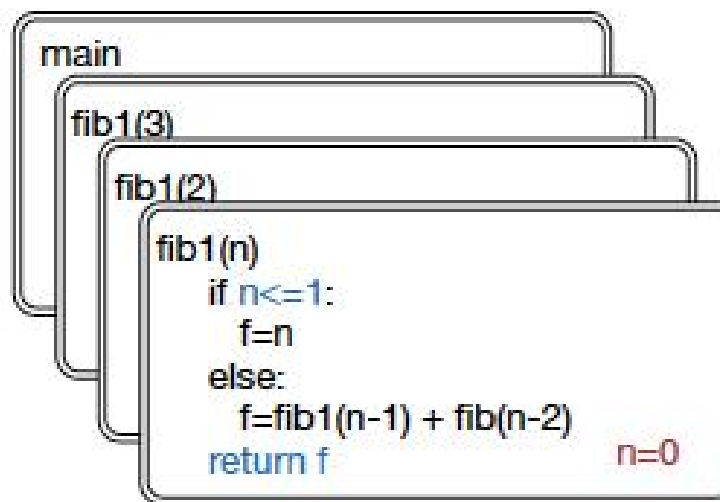




# 斐波那契序列



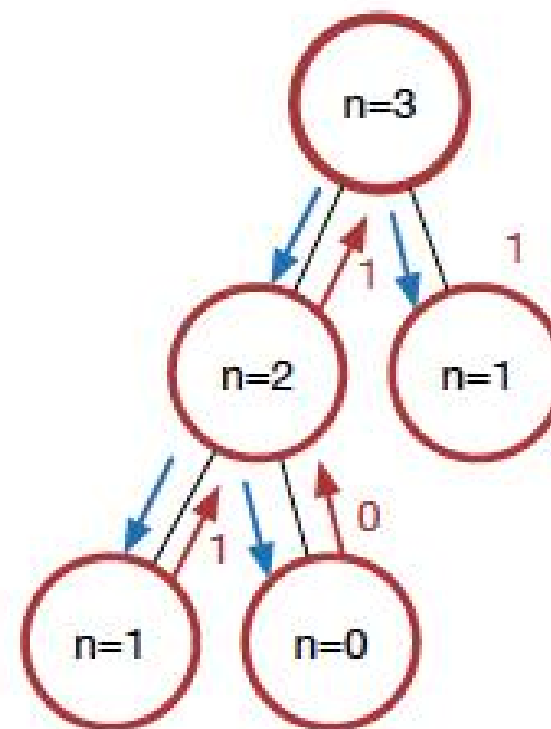
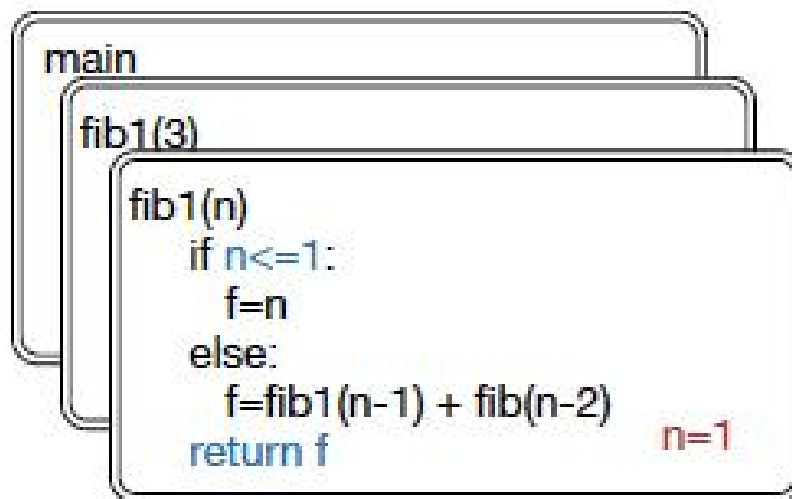
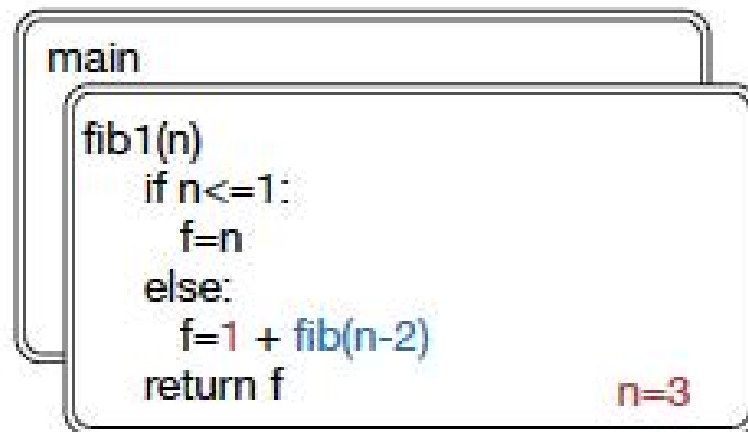
跟踪递归函数的  
执行过程



# 斐波那契序列



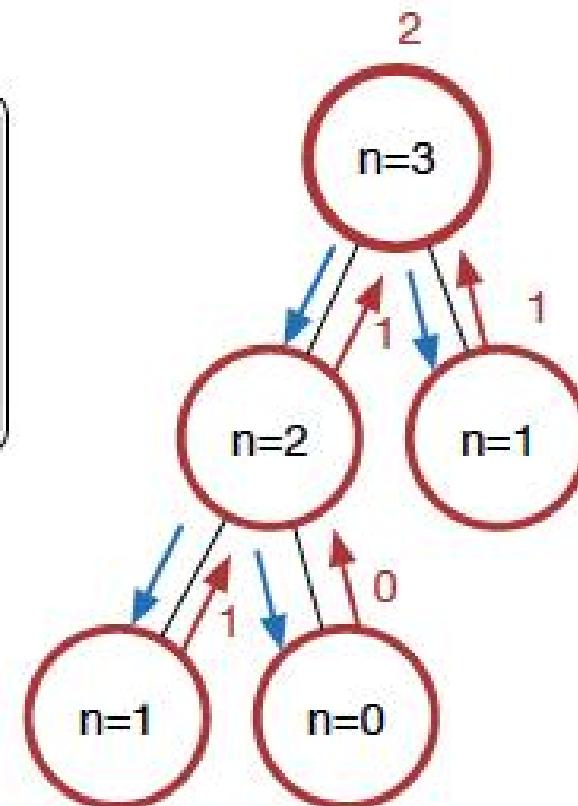
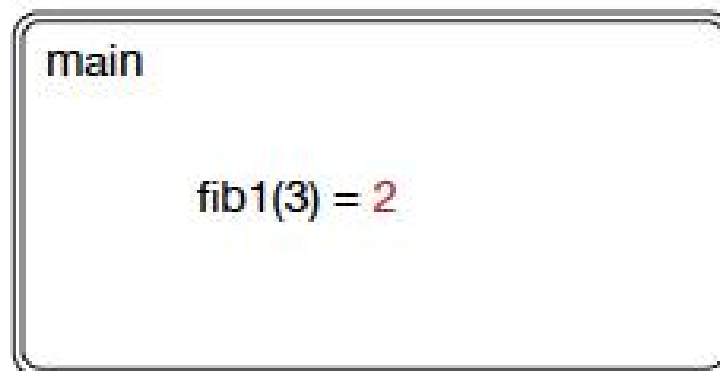
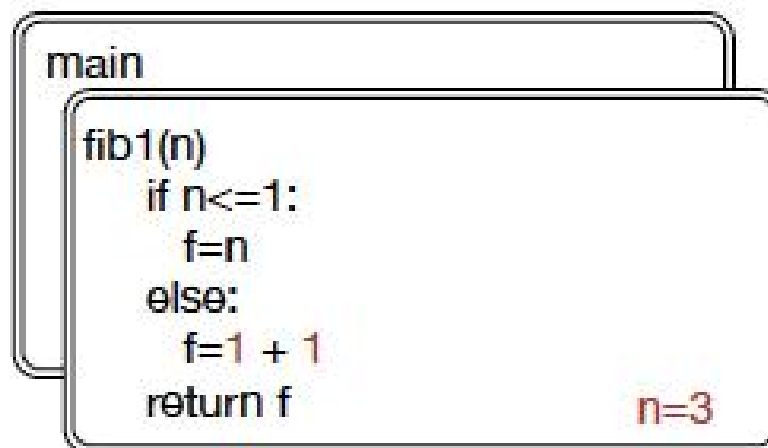
跟踪递归函数的  
执行过程



## 斐波那契序列



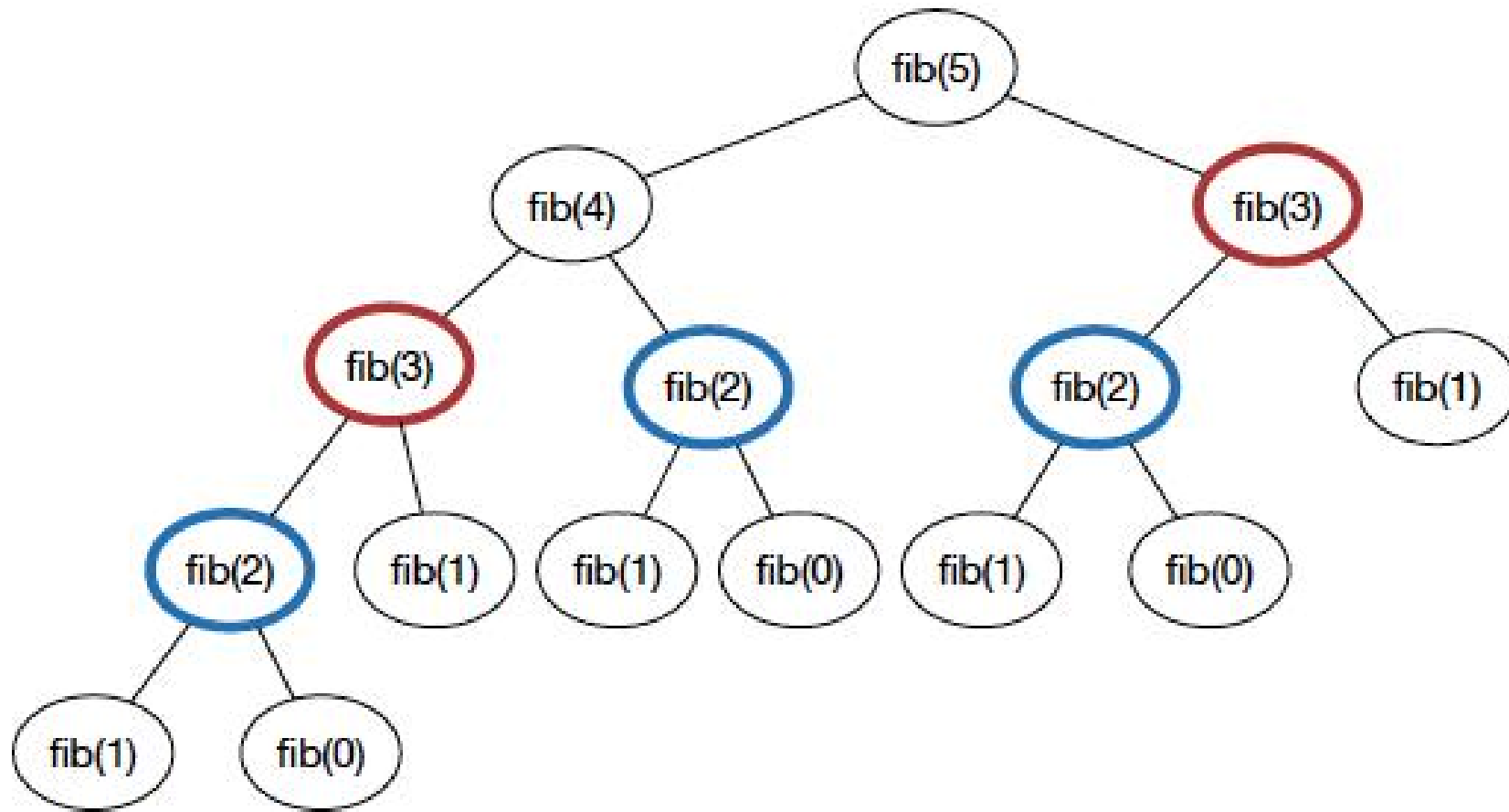
跟踪递归函数的  
执行过程



## 斐波那契序列



求斐波那契序列算法的时间复杂度:  $O(\frac{1}{\sqrt{5}} [(\frac{1+\sqrt{5}}{2})^n - (\frac{1-\sqrt{5}}{2})^n])$



## 斐波那契序列



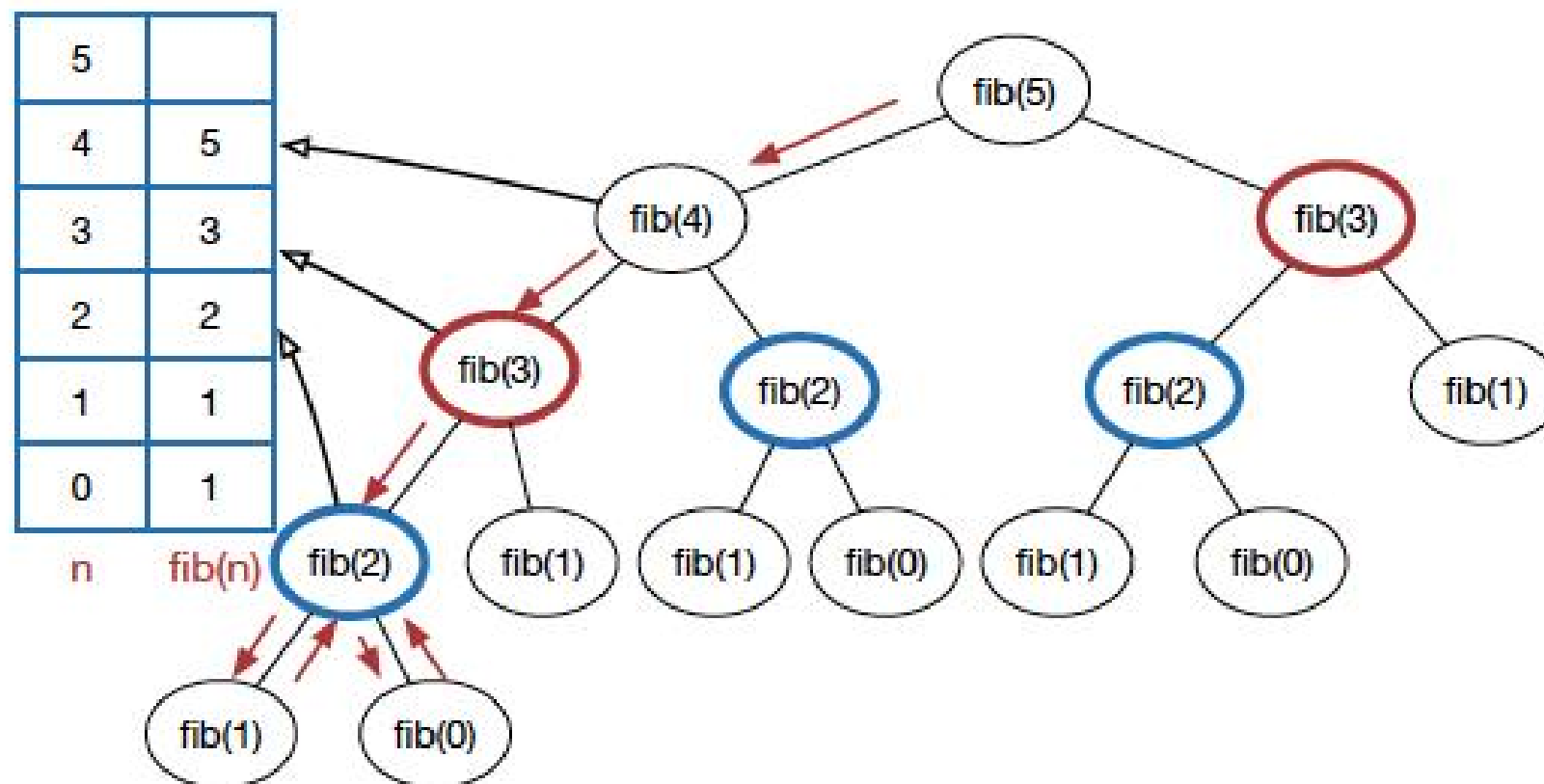
采用“记忆”减少递归调用

|   |   |   |   |   |   |   |    |    |    |    |    |     |
|---|---|---|---|---|---|---|----|----|----|----|----|-----|
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7  | 8  | 9  | 10 | 11 | 12  |
| 0 | 1 | 1 | 2 | 3 | 5 | 8 | 13 | 21 | 34 | 55 | 89 | 144 |

## 斐波那契序列



采用“记忆”减少递归调用



## 斐波那契序列



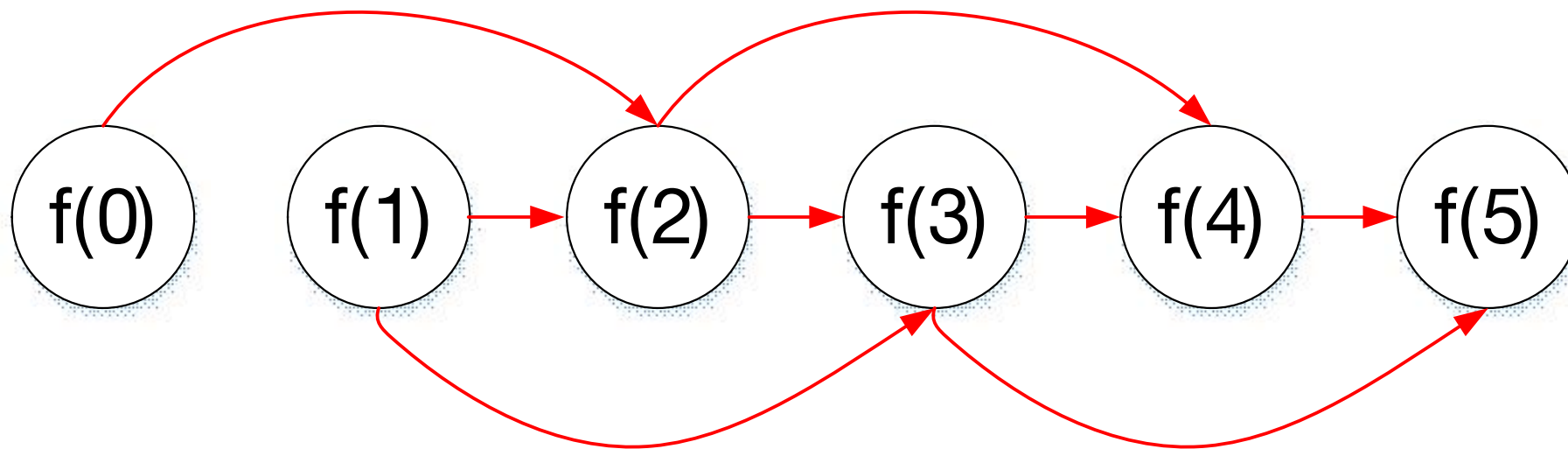
采用“记忆”减少递归调用  
自上而下的实现

```
memo = {}
def fib2(n):
    if n in memo:                # 查表
        return memo[n]
    else:
        if n <= 1:              # 边界条件
            f = n
        else:
            f = fib2(n-1) + fib2(n-2) # 递归调用
        memo[n] = f             # 将结果存储于表中
    return f
```

## 斐波那契序列



采用“记忆”传递信息  
自底而上的实现



时间复杂度为 $O(n)$



## 斐波那契序列



采用“记忆”传递信息

自底而上的实现

```
def fib_bottom_up(n):  
    fib = {}                                # 存储结果的字典  
    for k in range(n+1):  
        if k <= 2:                          # 边界条件  
            f = 1  
        else:  
            f = fib[k-1] + fib[k-2]          # 自底向上填表  
        fib[k] = f  
    return fib[n]
```

时间复杂度为 $O(n)$

## 斐波那契序列



求解斐波那契数列问题可以转变为求解矩阵幂运算问题:

$$\begin{bmatrix} F(n) \\ F(n-1) \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^{n-1} \times \begin{bmatrix} F(1) \\ F(0) \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^{n-1} \times \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

快速幂下, 可以将复杂度降低为 $O(\log n)$

# 目录

## 第一节

斐波那契序列

## 第二节

动态规划法的  
设计思想

## 第三节

三角数塔问题

## 第四节

最大子段和（最  
大子数组）

## 第五节

适用动态规划法  
的场景

## 第六节

最长公共子序  
列

## 第七节

0-1背包问题

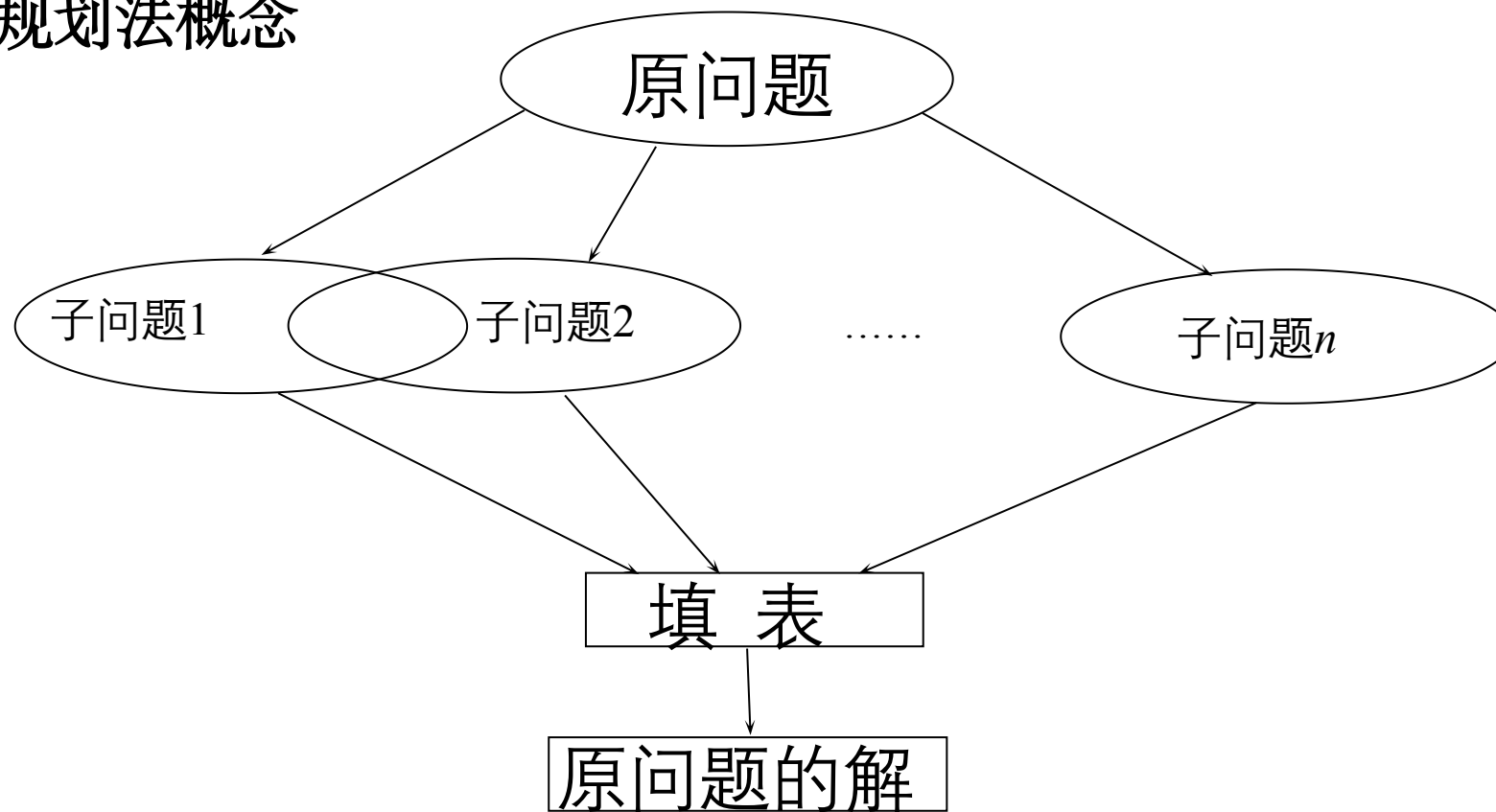
## 第八节

矩阵乘法

# 动态规划法的设计思想



## 动态规划法概念

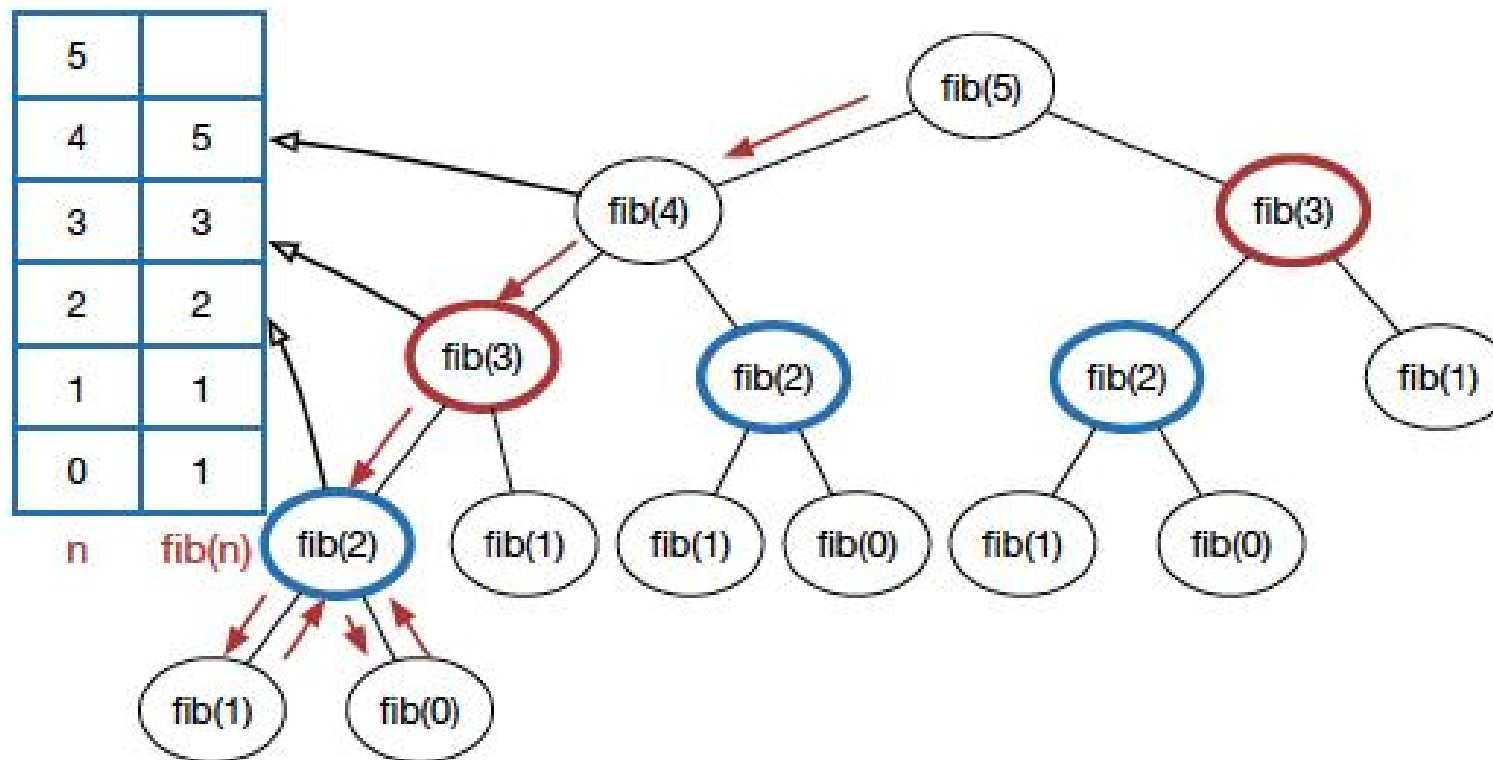


# 动态规划法的设计思想



Dynamic Programming

Programming如何翻译?



# 动态规划法的设计思想



## 动态规划法概念

动态规划是运筹学的一个分支，20世纪50年代初美国数学家Bellman等人在研究多阶段决策过程的优化问题时，提出了著名的最优性原理，创立了解决这类过程优化问题的新方法——动态规划法。



## 动态规划法的设计思想



动态规划法概念

$$F(n) = \begin{cases} 0 & n = 0 \\ 1 & n = 1 \\ F(n-1) + F(n-2) & n \geq 2 \end{cases}$$

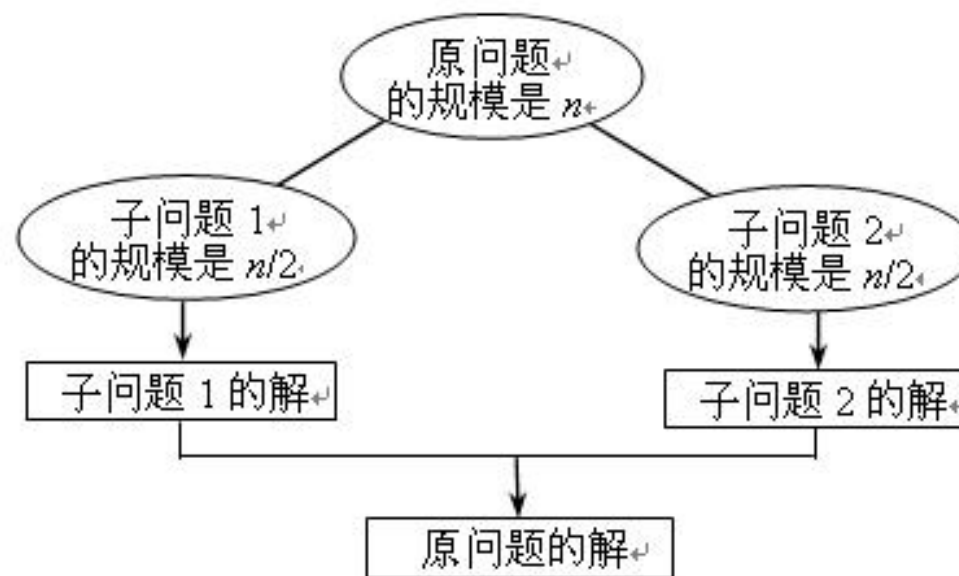
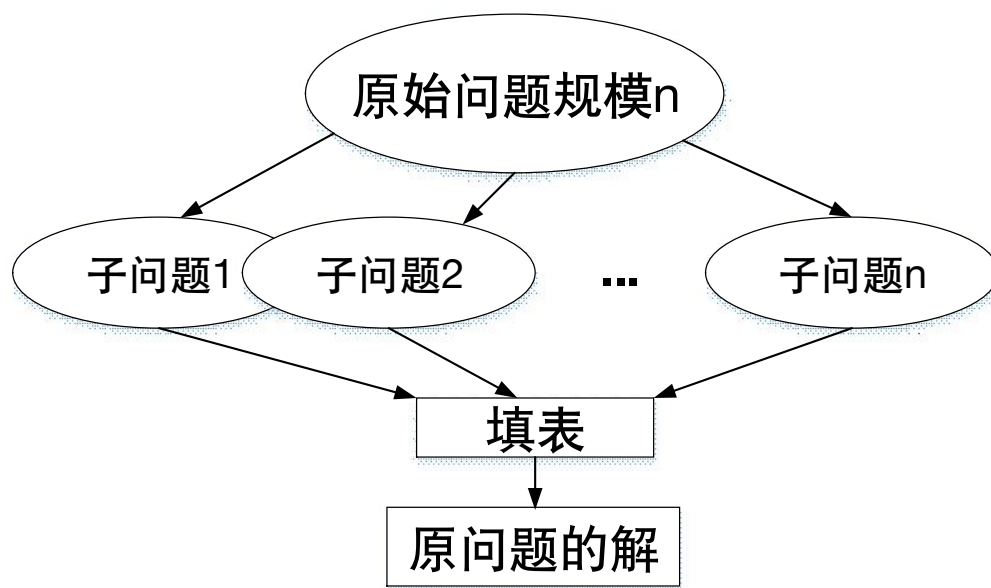
斐波那契序列

|   |   |   |   |   |   |   |    |    |    |    |    |     |
|---|---|---|---|---|---|---|----|----|----|----|----|-----|
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7  | 8  | 9  | 10 | 11 | 12  |
| 0 | 1 | 1 | 2 | 3 | 5 | 8 | 13 | 21 | 34 | 55 | 89 | 144 |

# 动态规划法的设计思想



## 动态规划法 vs 分治法







### 动态规划求解问题的5个步骤

1. 定义子问题
2. 建立各个子问题之间的递归关系（动态规划函数，数学模型）
3. 自底向上的求解递归式（或自顶向下）
4. 组合所有子问题的解从而获得原问题的解

## 动态规划法的设计思想



动态规划算法时间复杂度分析

子问题数  $\times$  每个子问题的单次求解时间

# 目录

## 第一节

斐波那契序列

## 第二节

动态规划法的设计思想

## 第三节

三角数塔问题

## 第四节

最大子段和（最大子数组）

## 第五节

适用动态规划法的场景

## 第六节

最长公共子序列

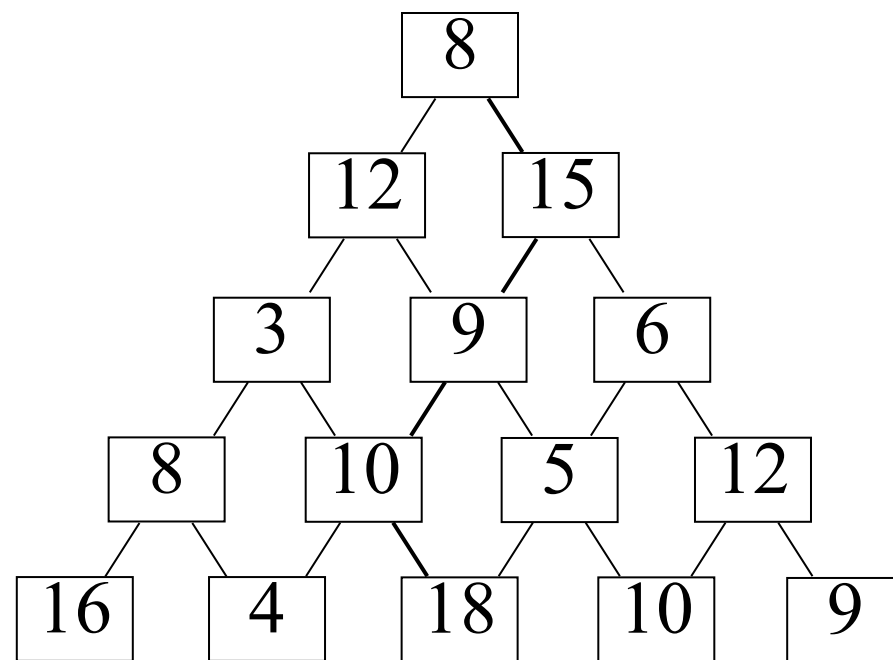
## 第七节

0-1背包问题

## 第八节

矩阵乘法

## 子问题重叠的例子：三角数塔问题

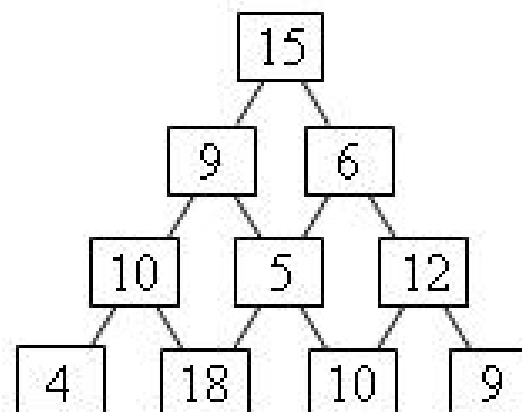
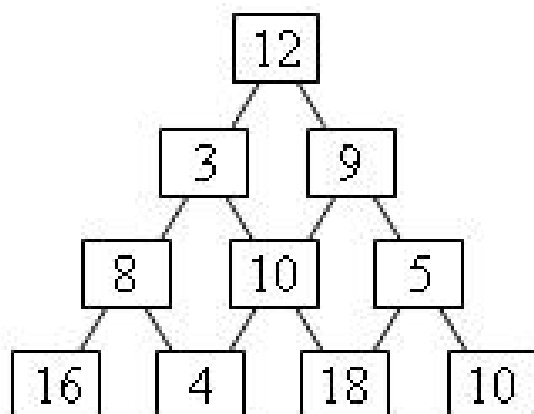
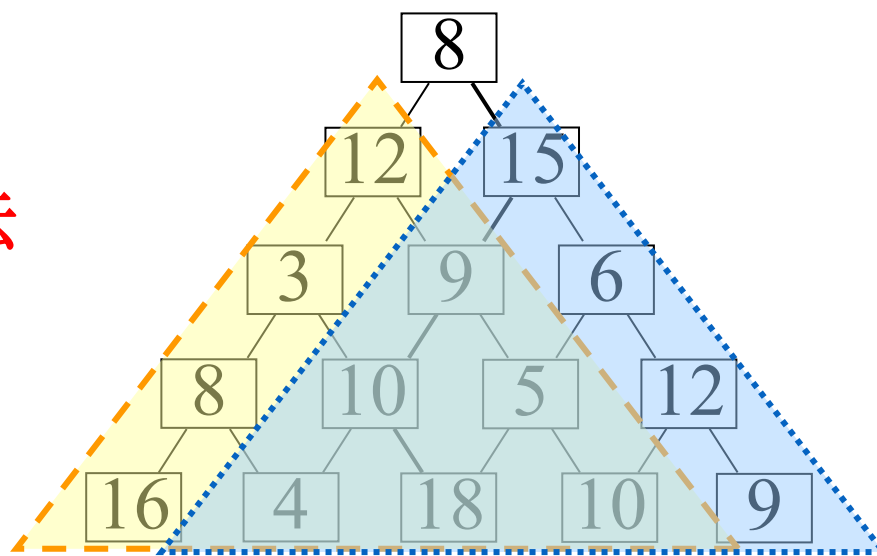


路径上的数值和最大

## 子问题重叠的例子：三角数塔问题



想法



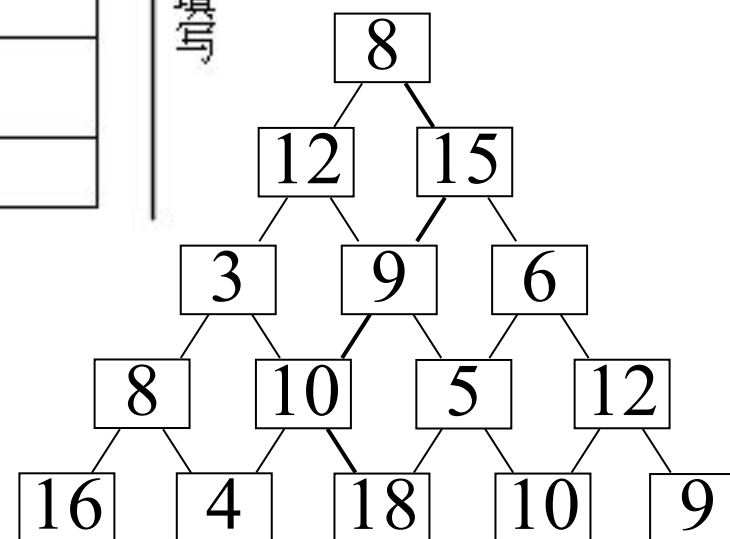
## 子问题重叠的例子：三角数塔问题



### 三角数塔决策过程

|          |                            |                            |                           |                           |   |
|----------|----------------------------|----------------------------|---------------------------|---------------------------|---|
| 第 1 层的决策 | $8 + \max\{49, 52\} = 60$  |                            |                           |                           |   |
| 第 2 层的决策 | $12 + \max\{31, 37\} = 49$ | $15 + \max\{37, 29\} = 52$ |                           |                           |   |
| 第 3 层的决策 | $3 + \max\{24, 28\} = 31$  | $9 + \max\{28, 23\} = 37$  | $6 + \max\{23, 22\} = 29$ |                           |   |
| 第 4 层的决策 | $8 + \max\{16, 4\} = 24$   | $10 + \max\{4, 18\} = 28$  | $5 + \max\{18, 10\} = 23$ | $12 + \max\{10, 9\} = 22$ |   |
| 初始化      | 16                         | 4                          | 18                        | 10                        | 9 |

自底向上填写



## 子问题重叠的例子：数塔问题



时间复杂度分析:  $O(n^2)$

# 目录

## 第一节

斐波那契序列

## 第二节

动态规划法的设计思想

## 第三节

三角数塔问题

## 第四节

最大子段和（最大子数组）

## 第五节

适用动态规划法的场景

## 第六节

最长公共子序列

## 第七节

0-1背包问题

## 第八节

矩阵乘法



## 最大子段和 (最大子数组)



### 最大子段和问题或最大子数组问题

给定由 $n$ 个整数（可能有负整数）组成的序列 $(a_1, a_2, \dots, a_n)$ ，最大子段和问题要求找到 $\sum_{k=i}^j a_k$ 的子序列 $(a_i, a_{i+1}, \dots, a_j)$  ( $1 \leq i \leq j \leq n$ )，当序列中所有整数均为负整数时，其最大子段和为0。例如，序列 $(-20, 11, -4, 13, -5, -2)$ 的最大子段和为 $\sum_{k=2}^4 a_k = 20$ 。

## 最大子段和 (最大子数组)



- 蛮力法: 时间复杂度

$$O(n^2)$$

- 分治算法: 时间复杂度

$$O(n \log n)$$

## 最大子段和（最大子数组）



动态规划: Step1 定义子问题

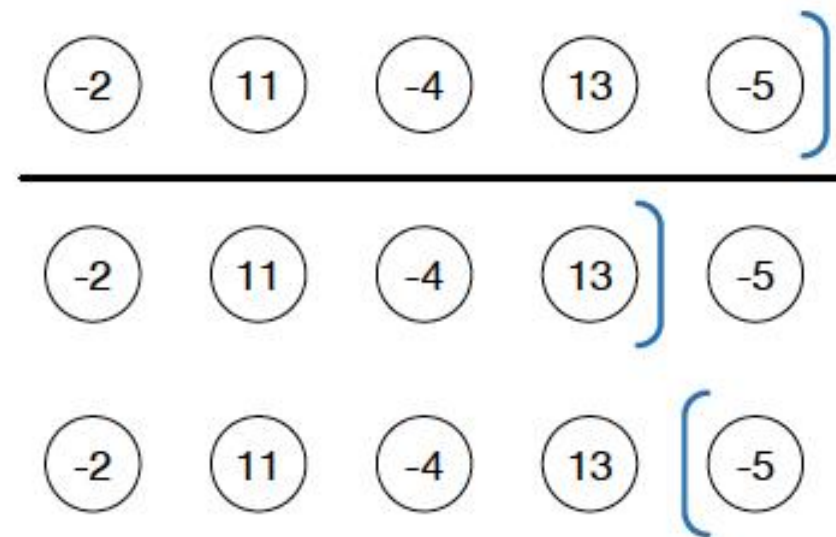
- 该问题输入的是一个有 $n$ 个元素的序列 $A$ , 不妨设 $P(i)$ 为直到第 $i$ 个元素的最大和
  - 将输入序列的前缀作为子问题

## 最大子段和 (最大子数组)



### 动态规划: Step2 猜测部分解

- 子问题 $P(i)$  的解包括第 $i-1$ 个元素, 此时有 $P(i)=P(i-1)+A[i]$
- 子问题 $P(i)$  的解不包括第 $i-1$ 个元素, 意味着需要从第 $i$  个元素开始重新计算值, 即 $P(i)=A[i]$



## 最大子段和 (最大子数组)



动态规划: Step3 建立各个子问题之间的动态规划函数

$$P(i) = \max\{P(i-1) + A[i], A[i]\}$$

初始条件为

$$P(0) = 0$$

## 最大子段和（最大子数组）



动态规划: Step4 自底向上的求解递归式

```
3 def bottom_up_cont_subseq(alist):
4     table = [None] * (len(alist) + 1)
5     table[0] = 0
6     for i in range(1, len(alist)+1):
7         table[i] = max(table[i-1] + alist[i-1], alist[i-1])
8     return table
```

• A=[-2, 11, -4, 13, -5, 2]

| $i$      | 0 | 1  | 2  | 3 | 4  | 5  | 6  |
|----------|---|----|----|---|----|----|----|
| table[i] | 0 | -2 | 11 | 7 | 20 | 15 | 17 |

时间复杂度:  $O(n)$

# 目录

## 第一节

斐波那契序列

## 第二节

动态规划法的设计思想

## 第三节

三角数塔问题

## 第四节

最大子段和（最大子数组）

## 第五节

适用动态规划法的场景

## 第六节

最长公共子序列

## 第七节

0-1背包问题

## 第八节

矩阵乘法

## 适用动态规划法的场景



什么情况下可以采用动态规划方法求解最优问题呢？

具备两个要素：

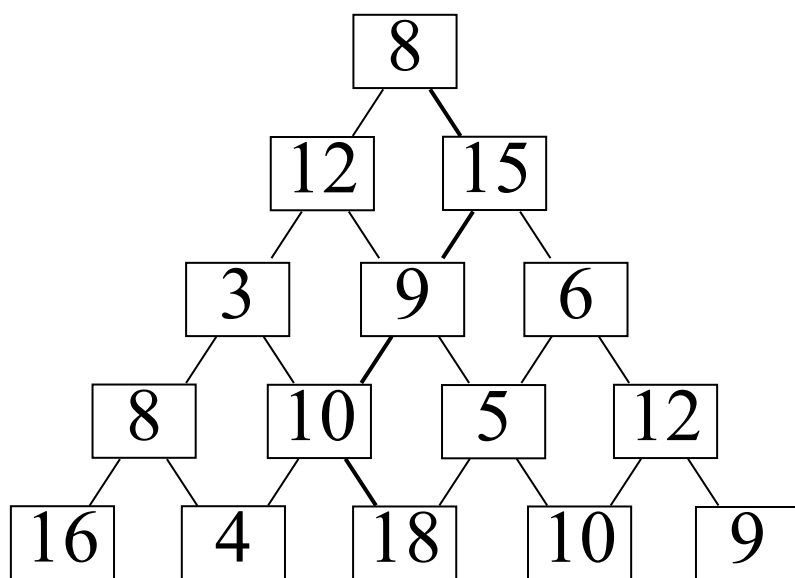
- (1) 最优子结构
- (2) 子问题重叠



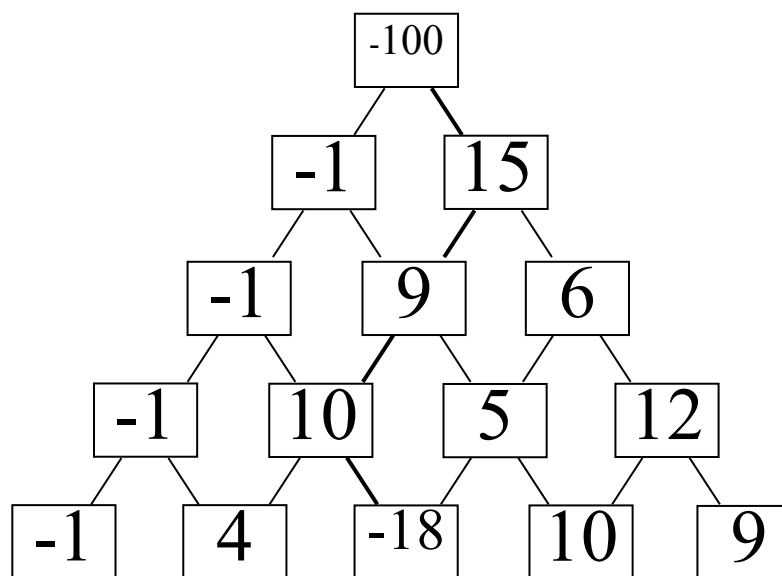
## 适用动态规划法的场景



### 最优子结构的反例：数塔问题



路径上的数值和最大



路径上的数值和的绝对值最大

## 适用动态规划法的场景



### 动态规划求解问题的5个步骤

1. 定义子问题
2. 猜测部分解 (刻画问题的最优子结构)
3. 建立各个子问题之间的递归关系 (动态规划函数, 数学模型)
4. 自底向上的求解递归式 (或自顶向下)
5. 组合所有子问题的解从而获得原问题的解

# 目录

## 第一节

斐波那契序列

## 第二节

动态规划法的设计思想

## 第三节

三角数塔问题

## 第四节

最大子段和（最大子数组）

## 第五节

适用动态规划法的场景

## 第六节

最长公共子序列

## 第七节

0-1背包问题

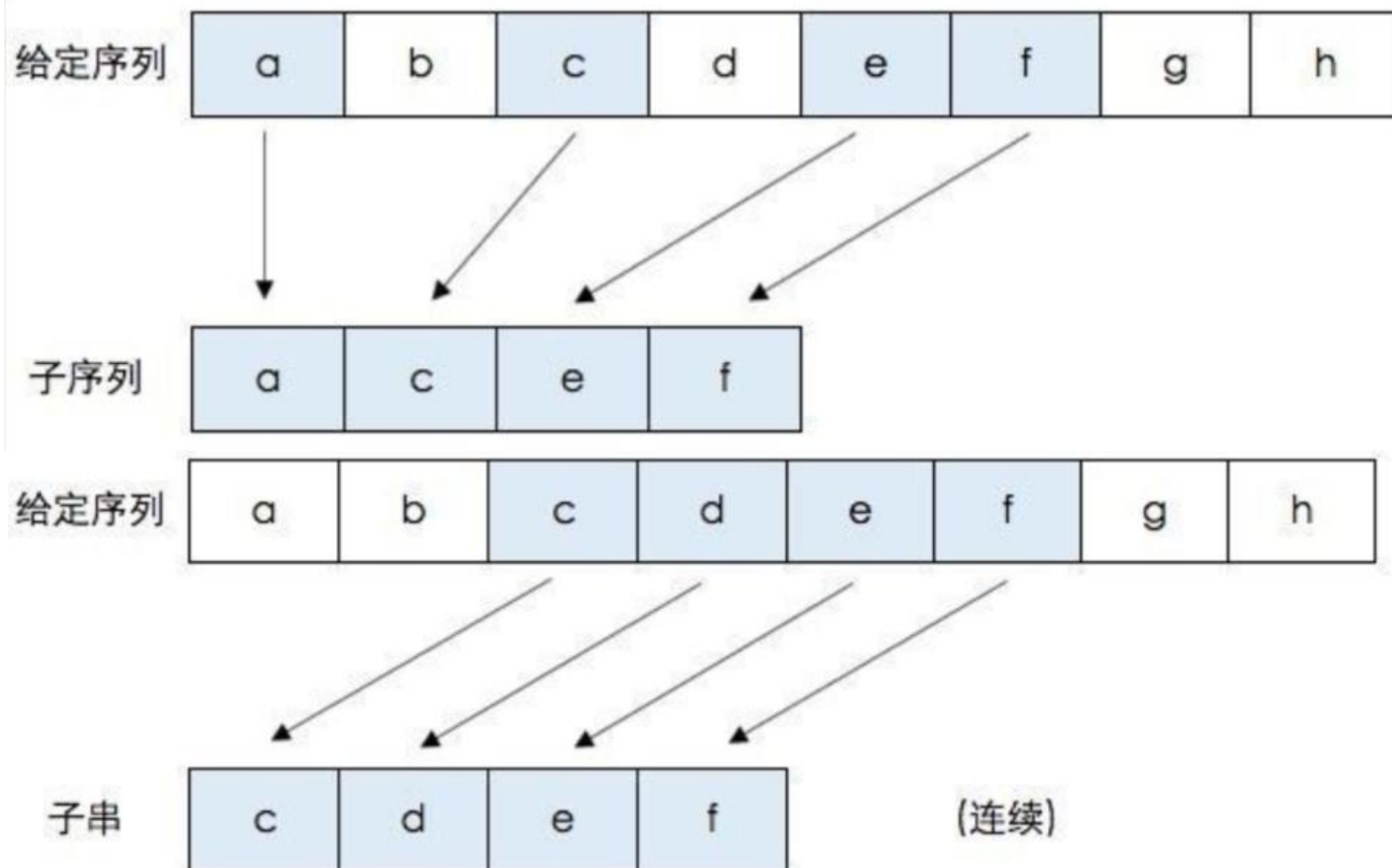
## 第八节

矩阵乘法

# 最长公共子序列



## 子序列与子串



## 最长公共子序列



### 问题描述:

对给定序列 $X=(x_1, x_2, \dots, x_m)$ 和序列 $Z=(z_1, z_2, \dots, z_k)$ ,  $Z$ 是 $X$ 的子序列当且仅当存在一个递增下标序列 $(i_1, i_2, \dots, i_k)$ , 使得对于所有 $j=1, 2, \dots, k$ , 有 $x_{i_j} = z_j$  ( $1 \leq i_j \leq m$ )。

给定两个序列 $X$ 和 $Y$ , 当序列 $Z$ 既是 $X$ 的子序列又是 $Y$ 的子序列时, 称 $Z$ 是序列 $X$ 和 $Y$ 的公共子序列。最长公共子序列问题就是在序列 $X$ 和 $Y$ 中查找最长的公共子序列。 Longest-common-subsequence:LCS

## 最长公共子序列



应用场景:

- 基因工程DNA
- 软件版本管理
- 软件自动测试
- 检查论文抄袭率
- 程序代码量相似度度量
- 视频段匹配

## 最长公共子序列



例子

X: 

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| A | B | C | B | D | A | B |
|---|---|---|---|---|---|---|

Y: 

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| B | D | C | A | B | A |
|---|---|---|---|---|---|

$Z_1$ : 

|   |   |   |   |
|---|---|---|---|
| B | D | A | B |
|---|---|---|---|

$Z_3$ : 

|   |   |   |   |
|---|---|---|---|
| B | C | B | A |
|---|---|---|---|

$Z_2$ : 

|   |   |   |   |
|---|---|---|---|
| B | C | A | B |
|---|---|---|---|

## 最长公共子序列



### 暴力搜索方法求解LCS问题

对于此问题，可以采用暴力求解的方式来比对，即穷举出X的所有子序列，用每个子序列与y做一一比较。假如X序列共有m个元素，对每个元素可以决定选或不选，则X的子序列个数共有 $2^m$ 个，可见与长度m呈指数阶，这种方法效率会很低。



## 最长公共子序列



### 问题步骤

第一步、给定序列  $X = \langle x_1, x_2, \dots, x_m \rangle$  和  $Y = \langle y_1, y_2, \dots, y_n \rangle$ , 求X 和Y 的最长公共子序列**长度**

第二步、给定序列  $X = \langle x_1, x_2, \dots, x_m \rangle$  和  $Y = \langle y_1, y_2, \dots, y_n \rangle$ , 求X 和Y 的最长公共子序列

## 最长公共子序列



动态规划: Step1 定义子问题

给定序列  $X = \langle x_1, x_2, \dots, x_m \rangle$  和  $Y = \langle y_1, y_2, \dots, y_n \rangle$ , 考虑X和Y的前缀, 定义  $C[i, j]$  为  $X_i = \langle x_1, x_2, \dots, x_i \rangle$  与  $Y_j = \langle y_1, y_2, \dots, y_j \rangle$  的最长公共子序列的长度

于是,  $C[m, n]$  就是X和Y的最长公共子序列的长度

## 最长公共子序列



### Step2:刻画最长公共子序列问题的最优子结构 (猜测部分解)

#### 定理15.1 (LCS的最优子结构)

设 $X_i = x_1x_2 \dots x_i$ 和 $Y_j = y_1y_2 \dots y_j$ 是两个序列,  $Z_k = z_1z_2 \dots z_k$ 是这两个序列的一个最长公共子序列。

- 1.如果 $x_i = y_j$ , 那么 $z_k = x_i = y_j$ , 且 $Z_{k-1}$ 是 $X_{i-1}$ ,  $Y_{j-1}$ 的一个最长公共子序列;
- 2.如果 $x_i \neq y_j$ , 那么 $Z_k$ 是 $X_{i-1}$ ,  $Y_j$ 的一个最长公共子序列或者 $Z_k$ 是 $X_i$ ,  $Y_{j-1}$ 的一个最长公共子序列。

从上面三种情况可以看出, 两个序列的LCS包含两个序列的前缀的LCS。因此, LCS问题具有最优子结构特征。



### Step2:刻画最长公共子序列问题的最优子结构 (猜测部分解)

#### 定理15.1 (LCS的最优子结构)

1.对  $X = \text{"AGTGATG"}, Y = \text{"GTTAG"}$ , 则  $Z = \text{"GTAG"}$ , 此时  $m = 7, n = 5, k = 4$  (从1开始)。

比较 $X$ 和 $Y$ 最后一位,  $x_7 = y_5$ , 同时删去,  $X_{m-1} = \text{"AGTGAT"}$ ,  $Y_{n-1} = \text{"GTТА"}$ ,  $Z_{k-1} = \text{"GTA"}$ 。我们可以发现,  $Z_{k-1}$ 就是 $X_{m-1}$ 和 $Y_{n-1}$ 的一个LCS。

2.对  $X = \text{"AGTGAT"}, Y = \text{"GTТА"}$ , 则  $Z = \text{"GTA"}$ , 此时  $m = 6, n = 4, k = 3$  (从1开始)。

比较最后一位,  $x_6 \neq y_4, z_3 \neq x_6$ ,  $X_{m-1} = \text{"AGTGA"}$ ,  $Z$ 是 $X_{m-1}$ 和 $Y_n$ 的一个LCS。

3.将2的 $X$ 和 $Y$ 互换即可

对  $Y = \text{"AGTGAT"}, X = \text{"GTТА"}$ , 则  $Z = \text{"GTA"}$ , 此时  $n = 6, m = 4, k = 3$  (从1开始)。

比较最后一位,  $x_4 \neq y_6, z_3 \neq x_4$ ,  $Y_{n-1} = \text{"AGTGA"}$ ,  $Z$ 是 $X_m$ 和 $Y_{n-1}$ 的一个LCS。

此定理说明两个序列的一个LCS也包含两个序列的前缀的一个LCS, 即LCS问题具有最优子结构性质。

## 最长公共子序列



Step3:建立各个子问题之间的动态规划函数

$$C[i, j] = \begin{cases} 0 & i = 0 \vee j = 0 \\ C[i - 1, j - 1] + 1 & i, j > 0 \wedge x_i = y_j \\ \max(C[i, j - 1], C[i - 1, j]) & i, j > 0 \wedge x_i \neq y_j \end{cases}$$

## 最长公共子序列



Step4: 自底向上的求解递归式(伪代码, 计算LCS长度)

LCS-LENGTH( $X, Y$ )

```
1   $m = X.length$ 
2   $n = Y.length$ 
3  let  $b[1..m, 1..n]$  and  $c[0..m, 0..n]$  be new tables
4  for  $i = 1$  to  $m$ 
5       $c[i, 0] = 0$ 
6  for  $j = 0$  to  $n$ 
7       $c[0, j] = 0$ 
8  for  $i = 1$  to  $m$ 
9      for  $j = 1$  to  $n$ 
10         if  $x_i == y_j$ 
11              $c[i, j] = c[i - 1, j - 1] + 1$ 
12              $b[i, j] = "\nwarrow"$ 
13         elseif  $c[i - 1, j] \geq c[i, j - 1]$ 
14              $c[i, j] = c[i - 1, j]$ 
15              $b[i, j] = "\uparrow"$ 
16         else  $c[i, j] = c[i, j - 1]$ 
17              $b[i, j] = "\leftarrow"$ 
18  return  $c$  and  $b$ 
```

|     |       | $j$   | 0 | 1               | 2                 | 3                 | 4                 | 5                 | 6                 |
|-----|-------|-------|---|-----------------|-------------------|-------------------|-------------------|-------------------|-------------------|
|     |       | $y_j$ |   | B               | D                 | C                 | A                 | B                 | A                 |
| $i$ | $x_i$ |       |   |                 |                   |                   |                   |                   |                   |
| 0   |       |       | 0 | 0               | 0                 | 0                 | 0                 | 0                 | 0                 |
| 1   | A     |       | 0 | $\uparrow$<br>0 | $\uparrow$<br>0   | $\uparrow$<br>0   | $\nwarrow$<br>1   | $\leftarrow$<br>1 | $\nwarrow$<br>1   |
| 2   | B     |       | 0 | $\nwarrow$<br>1 | $\leftarrow$<br>1 | $\leftarrow$<br>1 | $\uparrow$<br>1   | $\nwarrow$<br>2   | $\leftarrow$<br>2 |
| 3   | C     |       | 0 | $\uparrow$<br>1 | $\uparrow$<br>1   | $\nwarrow$<br>2   | $\leftarrow$<br>2 | $\uparrow$<br>2   | $\uparrow$<br>2   |
| 4   | B     |       | 0 | $\nwarrow$<br>1 | $\uparrow$<br>1   | $\uparrow$<br>2   | $\uparrow$<br>2   | $\nwarrow$<br>3   | $\leftarrow$<br>3 |
| 5   | D     |       | 0 | $\uparrow$<br>1 | $\nwarrow$<br>2   | $\uparrow$<br>2   | $\uparrow$<br>2   | $\nwarrow$<br>3   | $\uparrow$<br>3   |
| 6   | A     |       | 0 | $\uparrow$<br>1 | $\uparrow$<br>2   | $\uparrow$<br>2   | $\nwarrow$<br>3   | $\uparrow$<br>3   | $\nwarrow$<br>4   |
| 7   | B     |       | 0 | $\nwarrow$<br>1 | $\uparrow$<br>2   | $\uparrow$<br>2   | $\uparrow$<br>3   | $\nwarrow$<br>4   | $\uparrow$<br>4   |

## 最长公共子序列



Step4: 自底向上的求解递归式(伪代码, 计算LCS长度)

|   |   | A | B | C | B | D | A | B |
|---|---|---|---|---|---|---|---|---|
|   | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| B | 0 | 0 | 1 | 1 |   |   |   |   |
| D | 0 | 0 | 1 | 1 |   |   |   |   |
| C | 0 | 0 | 1 | 2 |   |   |   |   |
| A | 0 |   |   |   |   |   |   |   |
| B | 0 |   |   |   |   |   |   |   |
| A | 0 |   |   |   |   |   |   |   |

## 最长公共子序列



Step4: 自底向上的求解递归式(伪代码, 计算LCS长度)

|   |   | A | B | C | B | D | A | B |
|---|---|---|---|---|---|---|---|---|
|   | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| B | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 1 |
| D | 0 | 0 | 1 | 1 | 1 | 2 | 2 | 2 |
| C | 0 | 0 | 1 | 2 | 2 | 2 | 2 | 2 |
| A | 0 | 1 | 1 | 2 | 2 | 2 | 3 | 3 |
| B | 0 | 1 | 2 | 2 | 3 | 3 | 3 | 4 |
| A | 0 | 1 | 2 | 2 | 3 | 3 | 4 | 4 |



## 最长公共子序列



Step4: 自底向上的求解递归式(伪代码, 计算LCS长度)

|   |   | A | B | C | B | D | A | B |
|---|---|---|---|---|---|---|---|---|
|   | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| B | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 1 |
| D | 0 | 0 | 1 | 1 | 1 | 2 | 2 | 2 |
| C | 0 | 0 | 1 | 2 | 2 | 2 | 2 | 2 |
| A | 0 | 1 | 1 | 2 | 2 | 2 | 3 | 3 |
| B | 0 | 1 | 2 | 2 | 3 | 3 | 3 | 4 |
| A | 0 | 1 | 2 | 2 | 3 | 3 | 4 | 4 |

## 最长公共子序列



Step4: 自底向上的求解递归式(伪代码, 计算LCS长度)

```
PRINT-LCS( $b, X, i, j$ )
1  if  $i == 0$  or  $j == 0$ 
2      return
3  if  $b[i, j] == "\nwarrow"$ 
4      PRINT-LCS( $b, X, i - 1, j - 1$ )
5      print  $x_i$ 
6  elseif  $b[i, j] == "\uparrow"$ 
7      PRINT-LCS( $b, X, i - 1, j$ )
8  else PRINT-LCS( $b, X, i, j - 1$ )
```

|     |       | $j$ | 0                                                                                                                                                                             | 1               | 2               | 3               | 4               | 5                 | 6               |
|-----|-------|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|-----------------|-----------------|-----------------|-------------------|-----------------|
|     |       |     | $y_j$ <span><math>B</math></span> <span><math>D</math></span> <span><math>C</math></span> <span><math>A</math></span> <span><math>B</math></span> <span><math>A</math></span> |                 |                 |                 |                 |                   |                 |
| $i$ | $x_i$ |     |                                                                                                                                                                               |                 |                 |                 |                 |                   |                 |
| 0   |       |     | 0                                                                                                                                                                             | 0               | 0               | 0               | 0               | 0                 | 0               |
| 1   | $A$   |     | 0                                                                                                                                                                             | $\uparrow$<br>0 | $\uparrow$<br>0 | $\uparrow$<br>0 | $\nwarrow$<br>1 | $\leftarrow$<br>1 | $\nwarrow$<br>1 |
| 2   | $B$   |     | 0                                                                                                                                                                             | $\nwarrow$<br>1 | $\nwarrow$<br>1 | $\nwarrow$<br>1 | $\uparrow$<br>1 | $\nwarrow$<br>2   | $\nwarrow$<br>2 |
| 3   | $C$   |     | 0                                                                                                                                                                             | $\uparrow$<br>1 | $\uparrow$<br>1 | $\nwarrow$<br>2 | $\nwarrow$<br>2 | $\uparrow$<br>2   | $\uparrow$<br>2 |
| 4   | $B$   |     | 0                                                                                                                                                                             | $\nwarrow$<br>1 | $\uparrow$<br>1 | $\uparrow$<br>2 | $\uparrow$<br>2 | $\nwarrow$<br>3   | $\nwarrow$<br>3 |
| 5   | $D$   |     | 0                                                                                                                                                                             | $\uparrow$<br>1 | $\nwarrow$<br>2 | $\uparrow$<br>2 | $\uparrow$<br>2 | $\nwarrow$<br>3   | $\uparrow$<br>3 |
| 6   | $A$   |     | 0                                                                                                                                                                             | $\uparrow$<br>1 | $\uparrow$<br>2 | $\uparrow$<br>2 | $\nwarrow$<br>3 | $\uparrow$<br>3   | $\nwarrow$<br>4 |
| 7   | $B$   |     | 0                                                                                                                                                                             | $\nwarrow$<br>1 | $\uparrow$<br>2 | $\uparrow$<br>2 | $\uparrow$<br>3 | $\nwarrow$<br>4   | $\nwarrow$<br>4 |

## 最长公共子序列



时间复杂度:  $O(m \times n)$

空间复杂度:  $O(m \times n)$

# 目录

## 第一节

斐波那契序列

## 第二节

动态规划法的设计思想

## 第三节

三角数塔问题

## 第四节

最大子段和（最大子数组）

## 第五节

适用动态规划法的场景

## 第六节

最长公共子序列

## 第七节

0-1背包问题

## 第八节

矩阵乘法

## 0-1背包问题



- 假设有 $n$  个物品，每个物品 $i$  的价值为 $v_i$ ，大小为 $s_i$ 。包的容量为 $S$ ，要求从这 $n$  个物品中挑选若干物品装进包中，在所有装进包中物品的大小小于等于包容量 $S$  的前提下，包中物品总价值最大
- 例  $(s_0 = 3; v_0 = 4); (s_1 = 4; v_1 = 5); (s_2 = 5; v_2 = 6)$ 
  - 包的容量 $S = 10$ 。那么，应该选择物品 $(s_1 = 4; v_1 = 5); (s_2 = 5; v_2 = 6)$
  - $s_1 + s_2 = 9 \leq S = 10$ ，且这种选择的情况下总价值 $v_1 + v_2 = 11$  最大

## 0-1背包问题



### 贪心解法

- 迭代地选取 $v_i/s_i$ 最大的物品
- 这个算法无法得到最优解
- 反例:  $s_1 = 1, v_1 = 2; s_2 = S, v_2 = S$

## 0-1背包问题



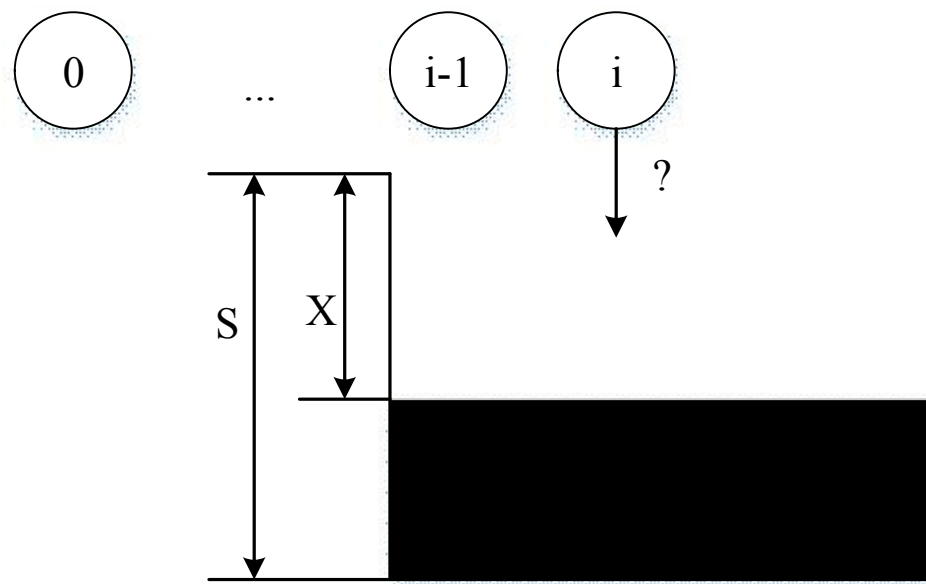
### step1 定义子问题

- 可以定义子问题从物品0、1、.....、 $i-1$ 、 $i$  中选取若干物品置于包中，这些物品的重量(容量)为 $X$ ，且获取了最佳收益，这个收益的值记为 $V(i, X)$ 。
- 于是， $V(n, S)$ 就是 $n$  个物品中挑选若干物品装进包中在所有装进包中物品的大小小于等于包容量 $S$  的前提下最大的包中物品总价值

## 0-1背包问题



### step2 刻画最优子结构



- ❑ 物品  $i$  不放入包中: 那么  $V(i, X)$  的值等于从物品,  $0, \dots, i-1$  选取容量为  $X$  的物品价值, 也就是  $V(i, X) = V(i-1, X)$ ;
- ❑ 物品  $i$  放入包中: 那么  $V(i, X)$  的值应该等于剩余物品,  $0, \dots, i-1$  放入容量为  $X-s_i$  的包中物品价值再加上物品  $i$  的价值  $v_i$ , 也就是  $V(i, X) = V(i-1, X-s_i) + v_i$ 。



## 0-1背包问题



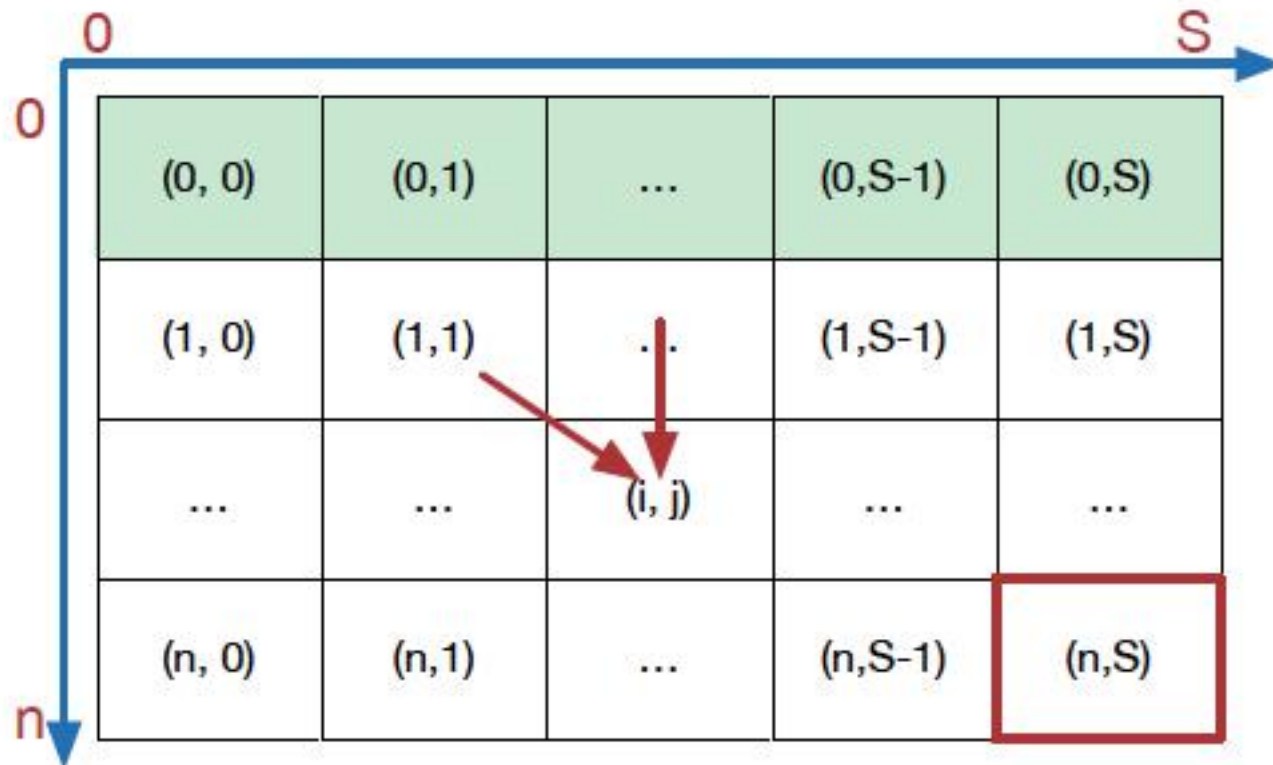
step3 建立各个子问题之间的递归关系 (动态规划函数)

1.  $X < s_i$  时,  
$$V(i, X) = V(i-1, X)$$
2.  $X \geq s_i$  时,  
$$V(i, X) = \max \{V(i-1, X-s_i) + v_i, V(i-1, X)\}$$

## 0-1背包问题



自底向上的求解递归式：填表



## 0-1背包问题



step3 建立各个子问题之间的递归关系 (动态规划函数)

初始化大小为  $n \times S$  数组  $V$

$$1) j(\text{or } X) < s(i) \quad V(i, j) = V(i-1, j)$$

$$2) j(\text{or } X) \geq s(i) \quad V(i, j) = \max \{ V(i-1, j), V(i-1, j-s(i)) + v(i) \}$$

$$V(i, 0) = V(0, j) = 0$$

## 0-1背包问题



**第一阶段**，只装入前1个物品，确定在各种情况下的背包能够得到的最大价值；

**第二阶段**，只装入前2个物品，确定在各种情况下的背包能够得到的最大价值；依此类推，直到第 $n$ 个阶段。

**最后**， $V(n, C)$ 便是在容量为 $C$ 的背包中装入 $n$ 个物品时取得的最大价值。为了确定装入背包的具体物品，从 $V(n, C)$ 的值向前推，如果 $V(n, C) > V(n-1, C)$ ，表明第 $n$ 个物品被装入背包，前 $n-1$ 个物品被装入容量为 $C-w_n$ 的背包中；否则，第 $n$ 个物品没有被装入背包，前 $n-1$ 个物品被装入容量为 $C$ 的背包中。

## 0-1背包问题



例子

eg:  $n = 4$ ,  $S = 8$

| i     | 1 | 2 | 3 | 4 |
|-------|---|---|---|---|
| s(大小) | 2 | 3 | 4 | 5 |
| v(价值) | 3 | 4 | 5 | 6 |

## 0-1背包问题



自底向上的求解递归式：填表

| i/j | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|-----|---|---|---|---|---|---|---|---|---|
| 0   | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 1   | 0 |   |   |   |   |   |   |   |   |
| 2   | 0 |   |   |   |   |   |   |   |   |
| 3   | 0 |   |   |   |   |   |   |   |   |
| 4   | 0 |   |   |   |   |   |   |   |   |

| i     | 1 | 2 | 3 | 4 |
|-------|---|---|---|---|
| s(大小) | 2 | 3 | 4 | 5 |
| v(价值) | 3 | 4 | 5 | 6 |

$$\begin{aligned}n &= 4 \\ S &= 8\end{aligned}$$

## 0-1背包问题



自底向上的求解递归式：填表

| i/j | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8  |
|-----|---|---|---|---|---|---|---|---|----|
| 0   | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0  |
| 1   | 0 | 0 | 3 | 3 | 3 | 3 | 3 | 3 | 3  |
| 2   | 0 | 0 | 3 | 4 | 4 | 7 | 7 | 7 | 7  |
| 3   | 0 | 0 | 3 | 4 | 5 | 7 | 8 | 9 | 9  |
| 4   | 0 | 0 | 3 | 4 | 5 | 7 | 8 | 9 | 10 |

| i     | 1 | 2 | 3 | 4 |
|-------|---|---|---|---|
| s(大小) | 2 | 3 | 4 | 5 |
| v(价值) | 3 | 4 | 5 | 6 |

$$\begin{aligned}n &= 4 \\ S &= 8\end{aligned}$$

## 0-1背包问题



从填表结果恢复最优解

| i/j | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8  |
|-----|---|---|---|---|---|---|---|---|----|
| 0   | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0  |
| 1   | 0 | 0 | 3 | 3 | 3 | 3 | 3 | 3 | 3  |
| 2   | 0 | 0 | 3 | 4 | 4 | 7 | 7 | 7 | 7  |
| 3   | 0 | 0 | 3 | 4 | 5 | 7 | 8 | 9 | 9  |
| 4   | 0 | 0 | 3 | 4 | 5 | 7 | 8 | 9 | 10 |

| i | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| s | 2 | 3 | 4 | 5 |
| v | 3 | 4 | 5 | 6 |

$V(i,j)=V(i-1,j)$ 时, 说明没有选择第 $i$ 个商品, 则回到 $V(i-1,j)$ ;

$V(i,j)=V(i-1,j-w(i))+v(i)$ 时, 说明装了第 $i$ 个商品, 该商品是最优解组成的一部分, 随后我们得回到装该商品之前, 即回到 $V(i-1,j-w(i))$ ;  
一直遍历到 $i=0$ 结束为止, 所有解的组成都会找到。



## 0-1背包问题



### 算法时间复杂度分析

时间代价 $O(n \times S)$ ，空间代价 $O(n \times S)$

问题：0-1背包问题的动态规划算法的时间复杂度是输入规模的多项式时间吗？

注意：这个不是多项式时间，因为 $S$ 是作为一个整数输入

设 $S$ 的位数是 $L = \log_2 S$ ，那么相当于时间代价和空间代价是 $O(n2^L)$

# 目录

## 第一节

斐波那契序列

## 第二节

动态规划法的设计思想

## 第三节

三角数塔问题

## 第四节

最大子段和（最大子数组）

## 第五节

适用动态规划法的场景

## 第六节

最长公共子序列

## 第七节

0-1背包问题

## 第八节

矩阵乘法

## 矩阵乘法



- 设 $A_1, A_2, \dots, A_n$ 为矩阵序列,  $A_i$ 为 $P_{i-1} \times P_i$ 维矩阵,  $i = 1, 2, \dots, n$ , 确定乘法顺序使得元素相乘的总次数最少
- 总的搜索空间大小为Catalan数, 即 $\frac{1}{n+1} \binom{2n}{n} = \Omega(4^n / n^{3/2})$

## 基本递归算法



RecurMatrixChain(P,i,j)

$m[i,j]=\infty$

for  $k=i$  to  $j-1$  do

$q = \text{RecurMatrixChain}(P,i,k) + \text{RecurMatrixChain}(P,k+1,j) +$

$p_{i-1} p_k p_j$

if  $q < m[i,j]$  then  $m[i,j]=q$

return  $m[i,j]$

时间复杂度 $O(2^n)$

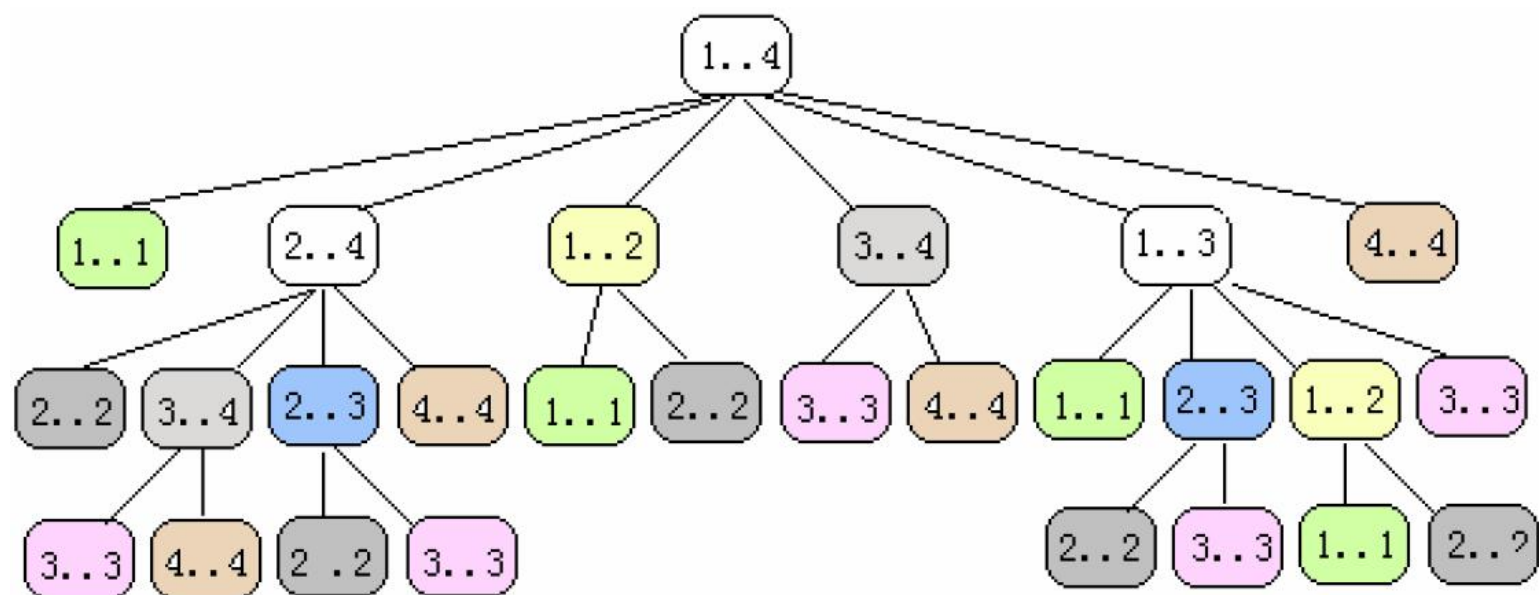
## 子问题定义



输入  $A = \langle A_0, A_1, \dots, A_n \rangle$ ,  $A_{i..j}$  表示乘积  $A_i A_{i+1} \dots A_j$  的结果, 其最后一次相乘是  $A_{i..j} = A_{i..k} A_{k+1..j}$

$m[i, j]$  表示得到  $A_{i..j}$  的最少的相乘次数, 于是有如下等式

$$m[i, j] = \begin{cases} 0, & \text{if } i = j \\ \min_{i \leq k < j} \{m[i, k] + m[k + 1, j] + A_{i-1} A_k A_j\}, & \text{if } i < j \end{cases}$$



## 基于动态规划的矩阵乘法



- MatrixChain(P,n)

for r=2 to n do // r为计算的矩阵链长度

for i =1 to n-r+1 do //n-r+1为最后r链的始位置

j =i+r-1 // 计算链i到j

$m[i,j] = m[i+1,j] + p_{i-1} * p_i * p_j$

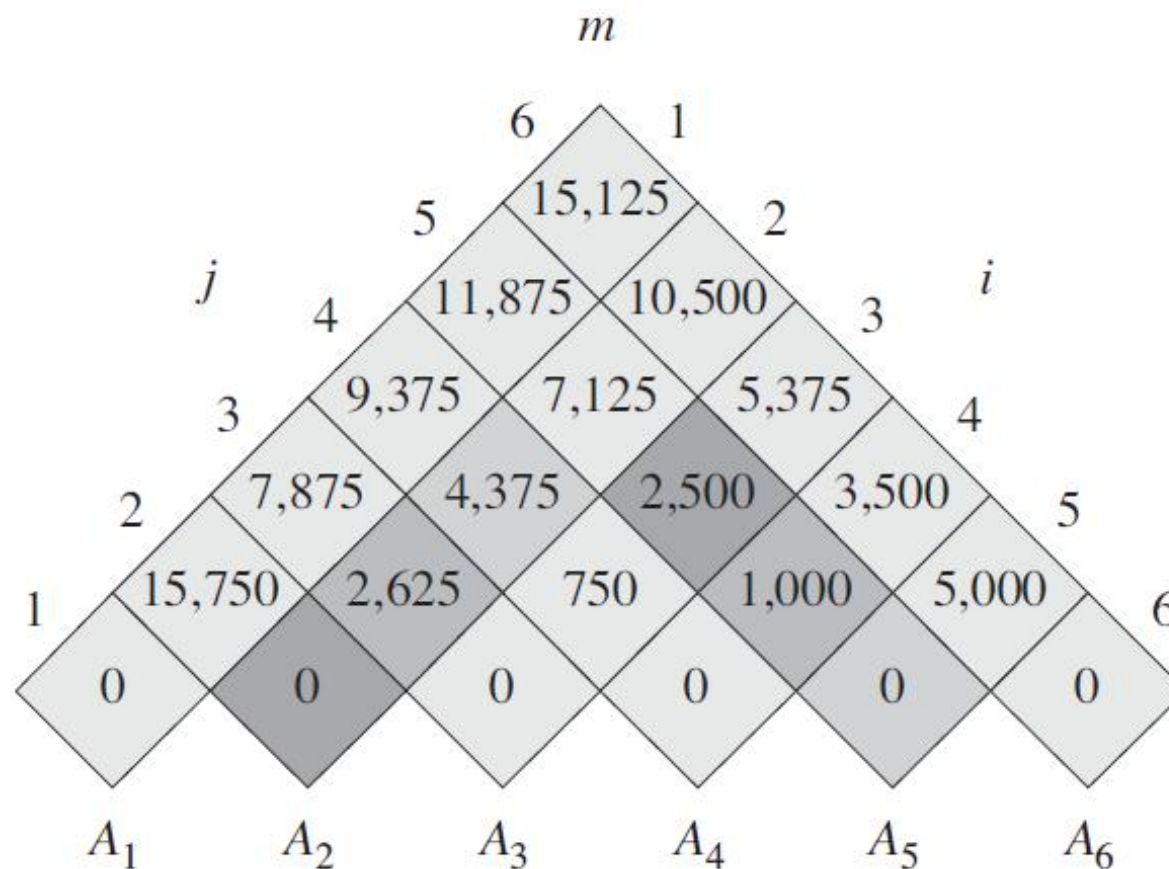
for k =i+1 to j-1 do

$t = m[i,k] + m[k+1,j] + p_{i-1} * p_k * p_j$

if  $t < m[i,j]$  then  $m[i,j] = t$

- 时间复杂度 $O(n^3)$

# 示例



| matrix    | $A_1$          | $A_2$          | $A_3$         | $A_4$         | $A_5$          | $A_6$          |
|-----------|----------------|----------------|---------------|---------------|----------------|----------------|
| dimension | $30 \times 35$ | $35 \times 15$ | $15 \times 5$ | $5 \times 10$ | $10 \times 20$ | $20 \times 25$ |





湖南大学  
HUNAN UNIVERSITY

谢谢观赏

—— 实事求是 敢为人先 ——